

ECDYSIS: Efficient and Effective Training of Runtime Harnesses for LLM Agents

Ruiqing Yue^{1,2*} Yu Cui^{3*} Zhuoyu Sun³ Sicheng Pan³ Xianhong Xue^{1,2}
 Tingyu Li³ Ting Li⁴ Wenzhuo Zhu⁴ Yi Chen^{1,2} Yifei Liu³
 Baohan Huang³ Zhe Cui^{1,2} Haibin Zhang^{5,6} Cong Zuo³

¹Chengdu Institute of Computer Applications, Chinese Academy of Sciences

²University of Chinese Academy of Sciences ³Beijing Institute of Technology

⁴Beijing University of Technology ⁵Yangtze Delta Region Institute of Tsinghua University, Zhejiang

⁶Jiaxing Key Laboratory of Artificial Intelligence and Cyber Resilience

<https://github.com/cuiyu-ai/Ecdysis>

Project Lead: Yu Cui <cuiyu@bit.edu.cn>

Abstract

Self-evolving runtime harnesses can substantially improve the capabilities of large language model (LLM) agents and provide a promising paradigm for optimizing agent execution. Existing harness evolution methods typically rely on iterative search, repeatedly evaluating and revising candidate harnesses based on execution feedback from task instances. While this paradigm enables continuous harness optimization, it incurs substantial time overhead due to repeated agent executions and code modifications, and may overfit to observed tasks and specific failure patterns, resulting in degraded generalization to unseen tasks. We identify the lack of principled failure diagnosis as a key bottleneck in harness evolution: an observed failure can reflect either model-specific deficiencies or systematic harness deficiencies, and directly optimizing against individual failures can lead to unnecessary model-specific accommodation. We therefore propose ECDYSIS, an efficient and effective framework that distinguishes model-specific accommodation from harness-level repair and biases adaptation toward systematic harness deficiencies by identifying recurring cross-task failure patterns. ECDYSIS adopts a batch-level cross-instance failure aggregation paradigm to jointly analyze failure evidence from multiple task instances and further introduces Failure-Driven Collaborative Refinement (FDCR) to diagnose failure causes and iteratively refine harness modification specifications. By combining cross-instance failure analysis with multi-role diagnosis, ECDYSIS enables more effective harness evolution with lower end-to-end training time. Experiments across multiple LLMs and benchmarks show that ECDYSIS achieves up to a $1.84\times$ speedup in harness training compared with existing harness evolution methods, while improving the reasoning accuracy of the resulting harnesses by 18.56%. Moreover, harnesses trained with ECDYSIS exhibit stronger generalization across LLMs and reduce inference-time token consumption.

1 Introduction

Large language models (LLMs) have demonstrated increasingly strong capabilities in complex reasoning. However, the capabilities of the underlying model alone are often insufficient for practical LLM agents [1]. Recent studies have shown that runtime harnesses can substantially enhance agent capabilities through inference-time mechanisms such as task planning, tool interaction, and

*Yu Cui proposed the algorithm, and Ruiqing Yue validated its effectiveness through experiments.

context management [2–4]. The widespread adoption of coding agents, such as Codex [5], further demonstrates that, even with the underlying LLM fixed, the runtime harness plays a crucial role in determining agent performance. Unlike manually designed harnesses, recent harness self-evolution methods [6–9] enable harnesses to automatically modify their own code based on execution feedback. By iteratively identifying failures and revising the harness, this paradigm provides a promising approach to automatically optimizing runtime harnesses for LLM agents.

Existing harness evolution methods typically rely on iterative search, repeatedly evaluating and revising candidate harnesses based on execution feedback collected from task instances. While this paradigm enables continuous optimization of runtime harnesses using task feedback, it faces substantial efficiency and generalization challenges. Each harness update typically requires additional agent execution, code modification, and result verification, resulting in substantial training-time overhead. Moreover, as evolution proceeds, the harness may become increasingly specialized to observed tasks and specific failure patterns, leading to undesirable deviations in harness evolution [10, 11] and degraded generalization to unseen tasks [12]. These limitations suggest that effective harness evolution requires not only collecting execution feedback, but also reliably interpreting such feedback and translating it into appropriate harness modifications.

We identify reliable failure attribution as a key challenge in harness self-evolution. An observed failure may arise from the current model’s behavior or from a systematic deficiency in the harness. This distinction is critical because harness evolution can either perform *model-specific accommodation*, adapting to idiosyncrasies of the current model, or *harness-level repair*, addressing systematic deficiencies in the interaction mechanism. Individual failures provide limited evidence for distinguishing these cases, and simply collecting more failures does not resolve the ambiguity. Without reliable attribution, evolution may overfit to observed failure patterns and degrade generalization to unseen tasks. We therefore seek to identify failure mechanisms that recur across task instances as stronger evidence for systematic harness repair.

This motivates the following research question: **Can failure evidence across task instances distinguish model-specific accommodation from systematic harness repair and guide more generalizable harness evolution?** To address this issue, we propose ECDYSIS, an efficient framework for evolving runtime harnesses for LLM agents. ECDYSIS adopts a batch-level cross-instance failure aggregation paradigm to jointly analyze failures from multiple task instances. Rather than treating failures as independent modification signals, it identifies recurring failure patterns across tasks and prioritizes those providing stronger evidence for systematic harness deficiencies.

To further improve the reliability of failure diagnosis, ECDYSIS introduces Failure-Driven Collaborative Refinement (FDCR) [13–15]. Different roles examine the collected failure evidence and iteratively refine potential failure causes and candidate harness modifications from complementary perspectives. A moderator then synthesizes these analyses into a structured harness modification specification, reducing ambiguity in failure interpretation and improving the reliability of the resulting modifications. By combining batch-level cross-instance failure analysis with multi-role diagnosis, ECDYSIS extracts more informative signals from each training round, prioritizing systematic harness repair over unnecessary model-specific accommodation and thereby enabling more effective harness evolution while reducing end-to-end training time.

We systematically evaluate ECDYSIS across multiple LLMs and benchmarks. Experimental results show that ECDYSIS achieves up to a $1.84\times$ speedup in harness training compared with existing harness evolution methods while improving the reasoning accuracy of the resulting harnesses by 18.56%. Further experiments demonstrate that harnesses trained with ECDYSIS exhibit stronger cross-LLM generalization, while also reducing inference-time token consumption. These results demonstrate that reliable failure diagnosis can improve both the efficiency and generalizability of runtime harness evolution. Our contributions are summarized as follows:

- **ECDYSIS Training Framework.** We propose ECDYSIS, a runtime harness training framework that analyzes recurring failure patterns across task instances to distinguish systematic harness deficiencies from model-specific behavior, and introduces Failure-Driven Collaborative Refinement (FDCR) to generate structured harness modification specifications.
- **Efficient and Generalizable Harness Evolution.** We systematically evaluate ECDYSIS across multiple LLMs and benchmarks. Results demonstrate substantial improvements in harness training efficiency and agent reasoning performance, together with stronger cross-model generalization and reduced token consumption.

- **Insights for Training Data Curation.** We propose a failure-aware training data selection strategy that considers interaction structures and failure mechanisms across task instances, prioritizing tasks that expose novel execution paths and harness deficiencies while reducing redundant failure signals. By aggregating failures across instances, ECDYSIS identifies shared deficiencies and enables more efficient harness evolution with fewer, more diagnostic training tasks. Our experiments show that ECDYSIS achieves performance comparable to full-data training while using only one-quarter of the training data, substantially reducing the overhead of harness evolution.
- **An Empirical Analysis of Model Accommodation in Harness Evolution.** Through extensive fine-grained manual analysis of the training process, we identify and quantify the tendency of failure-driven evolution to over-accommodate model-specific limitations using the model-accommodation ratio. Based on this analysis, we formulate a theoretical perspective in which excessive model accommodation can induce harness evolution drift and undermine cross-model generalization, providing an explanation for the improved generalization achieved by ECDYSIS.

2 Related Work

Inference-time mechanisms such as tool interaction, deliberate planning, and feedback-driven adaptation have substantially enhanced LLM agents [3, 16]. Building on these advances, automatic optimization has progressed from prompt optimization [17, 18] to agentic workflows and self-modifying agent programs [19–21]. More recently, several works have focused specifically on evolving runtime harnesses, using execution feedback and iterative evaluation to propose and validate harness modifications [22, 6, 23–26]. Although harnesses are ultimately optimized to improve LLM task performance, harness evolution should not simply adapt to every observed failure, as failures may reflect model limitations rather than harness deficiencies, leading to model-specific accommodation. ECDYSIS therefore treats recurrent failure patterns across independent task instances as stronger evidence of systematic harness deficiencies, guiding evolution toward reusable behavioral improvements rather than instance-specific adaptation.

3 ECDYSIS

3.1 Preliminary Analysis

The runtime harness can substantially influence task performance. Across six Qwen3 model-domain combinations, removing all harness layers yields an average accuracy of 29.72%, whereas a fixed, human-configured harness achieves 50.28%. Serially modifying the harness with a coding agent after each failure achieves 43.33%, which remains below the 50.28% achieved by the fixed human configuration. These results indicate that the runtime harness is a meaningful optimization target, while also suggesting that effective automatic harness evolution remains challenging. A more fundamental challenge lies in failure attribution: an observed failure may arise from the current model’s behavior or from a systematic harness deficiency. The former leads to *model-specific accommodation*, whereas the latter calls for *harness-level repair*. Individual failures provide limited evidence for distinguishing these cases, while treating failures from a common harness issue as independent modification signals can also incur redundant modification and validation costs.

This distinction motivates a conceptual decomposition of harness adaptation. Consider the aggregate harness update ΔH produced over an evolution round, composed of a set of independent modification decisions issued by the coding agent. Each decision either accommodates a limitation of the current task model or repairs a systematic harness deficiency, contributing to ΔH_{model} and $\Delta H_{\text{harness}}$, respectively. Letting $t \in [0, 1]$ denote the proportion of these decisions that are model-specific accommodation, the aggregate update decomposes as

$$\Delta H = t \Delta H_{\text{model}} + (1 - t) \Delta H_{\text{harness}}, \quad t \in [0, 1],$$

so that t is at once the fraction of model-accommodating decisions and the weight of the model-accommodation component in the overall update. A larger t may resolve observed failures efficiently but can overfit to the current model and training tasks. We hypothesize that failure patterns recurring across distinct task instances provide stronger evidence for systematic harness deficiencies than isolated failures, and therefore can shift adaptation away from unnecessary model-specific accommodation. Cross-task recurrence is used as an inductive bias rather than as proof of causal attribution.

Algorithm 1: ECDYSIS Training for Harness Self-Evolution

Input: Fixed task model θ , runtime environment $\mathcal{E}$, training task set $\mathcal{D}_{\text{train}}$, initial harness H_{base} , failure threshold λ , number of evolution rounds R , and number of refinement passes K

Output: Frozen final harness H_F

```
1  $H_0 \leftarrow H_{\text{base}};$ 
2 for  $i = 1$  to  $R$  do
3    $\mathcal{T}_i \leftarrow \text{Collect}(\theta, H_{i-1}, \mathcal{E}, \mathcal{D}_{\text{train}})$  # execution trajectories;
4    $f_\lambda(\tau) \leftarrow \mathbb{I}[S(\tau) < \lambda];$ 
5    $\mathcal{B}_i \leftarrow \text{Aggregate}(\{\tau \in \mathcal{T}_i : f_\lambda(\tau) = 1\})$  # structured failure evidence;
6   if  $\mathcal{B}_i = \emptyset$  then
7      $H_i \leftarrow H_{i-1};$ 
8     continue;
9    $\mathcal{G}_i \leftarrow \text{Group}(\mathcal{B}_i)$  # failure patterns;
10   $\mathcal{A} \leftarrow \{\text{Analyst, Critic, Engineer}\};$ 
11   $\mathcal{M}_i \leftarrow \emptyset$  # shared role transcript;
12  for  $k = 1$  to  $K$  do
13    foreach  $A \in \mathcal{A}$  do
14       $m_i^{k,A} \leftarrow A(\mathcal{G}_i, \mathcal{M}_i);$ 
15       $\mathcal{M}_i \leftarrow \mathcal{M}_i \circ m_i^{k,A};$ 
16   $q_i \leftarrow \text{Moderator}(\mathcal{G}_i, \mathcal{M}_i)$  # structured modification specification;
17   $H_i^c \leftarrow \text{Edit}(H_{i-1}, q_i)$  # coding agent;
18  if  $J_{\text{train}}(H_i^c) > J_{\text{train}}(H_{i-1})$  then
19     $H_i \leftarrow H_i^c;$ 
20  else
21     $H_i \leftarrow H_{i-1};$ 
22  $H_F \leftarrow H_R;$ 
23 return  $H_F;$ 
```

3.2 Methodology

Given a task model θ , a runtime environment $\mathcal{E}$, a training task set $\mathcal{D}_{\text{train}}$, and an initial runtime harness H_{base} , ECDYSIS optimizes the harness using execution trajectories collected from the training tasks. Throughout training, the task model parameters and runtime environment remain fixed. We initialize the harness as $H_0 = H_{\text{base}}$ and denote by H_i the validated harness retained after evolution round i .

For each evolution round $i \in \{1, 2, \dots, R\}$, ECDYSIS proceeds in three stages. First, it executes the training tasks with the current harness H_{i-1} and aggregates the resulting failure evidence. Second, it transforms the aggregated evidence into a structured modification specification, which is provided to a coding agent to modify H_{i-1} and produce a candidate harness H_i^c . Third, it validates H_i^c on the training set and retains it only if it improves the overall training score; otherwise, the previous harness is preserved. After R rounds, training terminates and ECDYSIS freezes the most recently validated harness, denoted by $H_F = H_R$. The complete procedure is summarized in Algorithm 1.

Let $J_{\text{train}}(H)$ denote the overall score of harness H on the training set under the fixed evaluation framework. In evolution round i , ECDYSIS accepts the candidate harness if and only if

$$J_{\text{train}}(H_i^c) > J_{\text{train}}(H_{i-1}).$$

Accordingly,

$$H_i = \begin{cases} H_i^c, & \text{if } J_{\text{train}}(H_i^c) > J_{\text{train}}(H_{i-1}), \\ H_{i-1}, & \text{otherwise.} \end{cases}$$

This criterion constrains only the overall training-set score and does not require every individual training task to improve. We next describe the two core components of ECDYSIS: Batch-Level Failure Aggregation and Failure-Driven Collaborative Refinement (FDCR).

3.3 Batch-Level Failure Aggregation

At evolution round i , ECDYSIS executes the training tasks using the current harness H_{i-1} and collects the resulting execution trajectories, denoted by $\mathcal{T}_i$. A task specifies the target objective, whereas its execution trajectory records the concrete process leading to the observed outcome. The trajectory therefore provides execution-level evidence for failure analysis beyond the final task-level signal.

For each trajectory $\tau \in \mathcal{T}_i$, the fixed evaluation framework produces a task score $S(\tau)$. We define the binary failure signal as

$$f_\lambda(\tau) = \mathbb{I}[S(\tau) < \lambda],$$

where λ is a predefined threshold and $f_\lambda(\tau) = 1$ indicates failure. Trajectories satisfying this condition are converted into structured records to form the failure evidence set $\mathcal{B}_i$. Each record retains the task identifier, failure decision, termination reason, tool-call history, and necessary execution context. This procedure uses the fixed evaluation framework as the training signal and does not modify or replace its scoring mechanism.

The failure records are organized into failure groups to facilitate the analysis of recurring patterns. ECDYSIS prioritizes groups that cover at least two distinct tasks, since repeated failures from a single task alone do not establish that the underlying failure mechanism generalizes across tasks. Groups containing only one task identifier are nevertheless retained as auxiliary evidence for subsequent analysis. Importantly, failure groups serve as diagnostic evidence rather than mandatory repair targets. They help determine whether an observed pattern indicates a harness deficiency and guide decisions regarding the modification level, trigger conditions, modification scope, and safety constraints. Thus, a candidate harness need not address every observed failure group.

Batch-level aggregation also reduces repeated coding-agent calls for candidate harness modification. Let $n_i = |\mathcal{B}_i|$ denote the number of failure records collected in round i . Processing each failure independently would require n_i coding-agent calls, whereas ECDYSIS makes a single coding-agent call for each nonempty round. Hence,

$$N_{\text{round}} = \sum_{i=1}^R \mathbb{I}(n_i > 0) \leq \sum_{i=1}^R n_i = N_{\text{serial}}.$$

The inequality is strict whenever at least one evolution round contains multiple failure records.

3.4 Failure-Driven Collaborative Refinement

For each evolution round containing failure evidence, ECDYSIS applies Failure-Driven Collaborative Refinement (FDCR) to transform the aggregated failure patterns into a structured harness modification specification. Inspired by collaborative criticism and iterative refinement in multi-agent systems [13, 14], FDCR uses observed failure patterns as the driving evidence for role-based diagnosis and iterative refinement, rather than treating individual failures as independent modification requests.

FDCR separates failure analysis and modification planning from the actual harness implementation. Given the failure groups $\mathcal{G}_i$, the Analyst, Critic, and Engineer iteratively refine modification proposals through a shared transcript. The Analyst identifies potential harness deficiencies and proposes minimal, targeted updates. The Critic evaluates these proposals against the observed failure evidence and examines potential risks, including overly broad triggers, unintended blocking of legitimate behavior, violations of the runtime contract, and regressions on previously successful tasks. The Engineer tracks agreements and unresolved disagreements and identifies issues requiring clarification in subsequent refinement. The roles are executed sequentially, with each role receiving the accumulated transcript so that subsequent analysis can build on preceding discussion. The refinement proceeds for a fixed number of rounds.

After the role-based refinement, the Moderator reads the failure evidence and the complete transcript and produces a structured modification specification. The Moderator serves as a conservative arbitration stage that consolidates the refined proposals, resolves remaining disagreements, and prioritizes modifications that address recurring failure patterns while avoiding unnecessary changes.

The resulting specification describes the identified failure patterns, proposed harness changes, and relevant implementation guidance, but does not directly modify the harness. Instead, the coding agent uses the specification together with the relevant execution evidence and the source code of the current harness to modify H_{i-1} and produce the candidate harness H_i^c .

FDCR and the coding agent therefore have distinct responsibilities: FDCR diagnoses cross-instance failure patterns and refines the corresponding modification specification, whereas the coding agent performs the actual harness modification. This separation keeps failure analysis structured and failure-driven while delegating implementation to the coding agent. By prioritizing failure patterns recurring across distinct task instances, FDCR biases the modification process toward systematic harness-level repairs and away from unnecessary model-specific accommodation.

3.5 Evaluation

We evaluate ECDYSIS from three perspectives: inference performance, training efficiency, and cost.

- **Inference Performance.** On the held-out split, we report *average task accuracy*, *Pass@3*, and *Pass^3*. *Pass@3* denotes the proportion of tasks solved in at least one of the three trials, whereas *Pass^3* denotes the proportion solved in all three trials.
- **Training Efficiency.** *Training Time* measures the end-to-end wall-clock time from the initial training evaluation until the final harness is frozen. For held-out evaluation, we report *Final Evaluation Tokens*, defined as the total input and output tokens consumed by the task model.
- **Cost.** We report *Input Cache Hit Rate* and *API Cost*. *Input Cache Hit Rate* is the ratio of cached input tokens to total input tokens. *API Cost* is reported separately for training task evaluation and harness modification.

4 Experiments

4.1 Experimental Setup

Models. Our evaluation follows prior work [27] and considers the compatibility between LLM reasoning capabilities and benchmark difficulty. We evaluate five task LLMs: Qwen3-8B, Qwen3-14B, Qwen3-32B [28], MiniMax-M2.7 (230B)², and Llama-3.1-8B³. For reproducibility, all LLMs are accessed through APIs. We use the same inference configuration for all task models in both training and held-out evaluation. For evaluation parameters, we refer to the baseline methods. The sampling temperature is set to 0.0. We use OpenCode⁴ with DeepSeek-V4-Pro as its underlying model as the coding agent. In ECDYSIS (w/ FDCR), the Analyst, Critic, Engineer, and Moderator also use DeepSeek-V4-Pro [29].

Datasets. To ensure comprehensive coverage of diverse task types, we use AgentBench [30] for relatively simple tasks and τ^2 -Bench for more complex and challenging tasks. Following prior work, we use the Airline and Retail subsets of τ^2 -Bench, which we refer to as τ^2 -Airline and τ^2 -Retail, respectively. Both subsets require agents to perform tool interactions that continuously modify the environment state [31]. For each subset, we select 20 tasks from the training split and 20 tasks from the test split for all models and methods. For held-out evaluation, we evaluate each test task in three independent trials, resetting the environment before each trial. Thus, each combination of model, subset, and method contains 60 held-out trajectories.

Baselines and Ablation Study. For the runtime harness, we adopt *Life-Harness*, a mature and well-structured harness framework [27]. To enable fair and controlled comparisons, we construct five runtime harness configurations, covering a direct baseline, a human-optimized baseline, and three harness evolution strategies.

- **Direct:** retains the base agent loop, message handling, and tool interface required by τ^2 -Bench, while disabling the harness layers adopted from Life-Harness. This configuration serves as the minimal baseline without any additional harness mechanisms.

²<https://huggingface.co/MiniMaxAI/MiniMax-M2.7>

³<https://huggingface.co/meta-llama/Llama-3.1-8B>

⁴<https://opencode.ai>

Table 1: Overall task performance and inference efficiency across five LLMs and two datasets. Inference efficiency is measured by token consumption (M) and execution time (s).

Method	Accuracy (%)			Efficiency	
	AVG	Pass@3	Pass^3	Token Cost	Time
Direct	38.17 $\pm$ 28.57	53.50	22.50	8.701 $\pm$ 4.509	87.91 $\pm$ 38.06
Human-Aug.	51.67 $\pm$ 24.10	66.00	37.00	11.349 $\pm$ 4.325	118.53 $\pm$ 66.11
Self-Evolution	46.67 $\pm$ 25.40	63.50	29.00	11.567 $\pm$ 5.161	126.08 $\pm$ 58.72
ECDYSIS (w/o FDCR)	54.67 $\pm$ 25.48	66.50	42.00	10.354 $\pm$ 4.166	131.42 $\pm$ 83.46
ECDYSIS (w/ FDCR)	59.33 $\pm$ 25.03	71.50	45.00	10.157 $\pm$ 3.152	118.69 $\pm$ 56.82

- **Human-Augmented Harness (Human-Aug.):** uses a fixed harness without harness evolution. This harness is obtained through human-involved optimization in prior work and serves as a strong baseline representing the performance of a manually optimized runtime harness. Moreover, the human-optimized harness is used as the common base harness for harness post-training. That is, all three evolution-based methods described below are initialized from the same Human-Aug. harness, thereby keeping the initial harness configuration fixed across methods.
- **Self-Evolution (SE):** performs instance-level serial updates. Each failure record independently invokes the coding agent, and the modifications produced for individual failure instances are applied sequentially and accumulated into a single round-level candidate. The accumulated candidate is then evaluated in the next complete training evaluation [27].
- **ECDYSIS (w/o FDCR):** performs mixed training by aggregating failure evidence from multiple training tasks and evolution rounds. For each evolution round with non-empty failure evidence, it constructs a single round-level modification plan from the aggregated evidence and invokes the coding agent once to implement the proposed changes.
- **ECDYSIS (w/ FDCR):** operates on the same mixed training input as ECDYSIS (w/o FDCR). The Analyst, Critic, and Engineer agents jointly analyze the aggregated failure evidence through two rounds of FDCR. The Moderator then synthesizes the results and formulates the final modification plan, which is subsequently implemented by the coding agent.

Across all three evolution-based methods, only the runtime harness is updated during training, while the model parameters remain fixed. Thus, the comparison isolates the effect of different harness optimization strategies while controlling for the initial harness configuration and the underlying model.

Evaluation Protocol. The three methods share the same training-time evaluation and candidate acceptance protocol, differing only in how they organize failure evidence and construct candidate modifications. We use Qwen3-8B as the task model and allow at most three candidate-generation rounds. Starting from the same initial harness, each method generates candidates from the resulting evidence, which are evaluated only in the subsequent complete evaluation and never on the trajectories used for their generation. A candidate is retained only if it strictly improves over the previously retained harness; otherwise, it is discarded. Accepted candidates are used for subsequent generation. After the final candidate is generated, a final evaluation over all training tasks is performed solely to determine whether it is accepted, with no further candidate generation. All evolution decisions are based exclusively on the training split. After evolution, the resulting harnesses are frozen and evaluated on held-out data under two independent protocols: first, Qwen3-8B is evaluated on all held-out tasks in each subset with three trials per task; second, we evaluate the full matrix of five task models, two subsets, and five methods, directly reusing the harnesses evolved with Qwen3-8B for the three evolution-based methods without further evolution.

5 Results

5.1 Overall Results

Across the ten combinations of five task models and two τ^2 -Bench subsets, ECDYSIS (w/ FDCR) achieves the highest average accuracy (Table 1). Its relative improvements over SE and Human-Aug. are 27.1% and 14.8%, respectively. It also improves Pass^3 by 55.2% relative to SE, showing more consistent success across repeated trials.

Beyond task accuracy, ECDYSIS also improves both training and inference efficiency relative to SE. We report the training results in Section 5.3 and the inference results in Section 6.2, and analyze the sources of these gains in Section 7.

5.2 Ablation Results

All three task-performance metrics improve from SE to ECDYSIS (w/o FDCR). The accuracy increases from 46.67% to 54.67%, an improvement of 8.00%. Meanwhile, Pass@3 and Pass³ reach 66.50% and 42.00%, improving by 3.00% and 13.00%, respectively. Adding FDCR further increases the average task success rate to 59.33%, Pass@3 to 71.50%, and Pass³ to 45.00%, corresponding to additional gains of approximately 4.67%, 5.00%, and 3.00% over ECDYSIS (w/o FDCR). These results show that round-level aggregation of failure evidence across task instances provides performance gains on its own, with FDCR further improving the aggregate results.

5.3 Training Efficiency

The training-efficiency results show that round-level failure aggregation reduces repeated modification calls during harness evolution. ECDYSIS (w/o FDCR) completes end-to-end training in 1,292.4 seconds on τ^2 -Retail and 2,510.8 seconds on τ^2 -Airline, corresponding to speedups of 1.42 $\times$ and 3.23 $\times$ over SE. ECDYSIS (w/ FDCR) takes 1,405.9 and 4,403.0 seconds, corresponding to speedups of 1.30 $\times$ and 1.84 $\times$ (Table 6). API cost follows the same pattern. On τ^2 -Retail and τ^2 -Airline, SE costs \$8.484 and \$6.382, whereas ECDYSIS (w/o FDCR) reduces these costs to \$2.485 and \$2.136, and ECDYSIS (w/ FDCR) costs \$5.763 and \$2.609, respectively (see Table 4). Overall, ECDYSIS decouples efficient failure-driven evolution from costly collaborative refinement. Failure aggregation provides the primary efficiency gains, while FDCR serves as an accuracy-oriented refinement module that trades additional evolution-time cost for improved harness quality.

6 Complete Evaluation Results

6.1 Task Performance

Averaged over the five task models and the three datasets (τ^2 -Airline, τ^2 -Retail, and AgentBench), the average accuracy increases from 58.67% under SE to 69.56% with ECDYSIS (w/ FDCR), a relative gain of 18.56%. Results on τ^2 -Airline and τ^2 -Retail are detailed in Table 2, and those on AgentBench are reported in Table 7. The improvement is especially pronounced for Qwen3-8B on the τ^2 -Airline subset, where accuracy rises from 35.00% to 60.00%, Pass@3 from 50.00% to 80.00%, and Pass³ from 20.00% to 40.00%. Notably, this improvement extends to models not used during evolution. On the same subset, Qwen3-32B improves from 51.67% under SE to 68.33% with ECDYSIS. These results demonstrate that a harness evolved with Qwen3-8B transfers to other task models without further evolution.

6.2 Inference Efficiency

We report token consumption and mean per-trajectory runtime for the complete held-out matrix in Table 3. Final evaluation tokens include only the input and output tokens consumed by the task model and user simulator during the formal held-out trajectories. Runtime is measured per trajectory rather than as the wall-clock duration of the concurrent evaluation.

Averaged over the ten model and subset combinations, final evaluation tokens are 11.57M for SE, 10.35M for ECDYSIS (w/o FDCR), and 10.16M for ECDYSIS (w/ FDCR), corresponding to relative reductions of 10.48% and 12.19%. This indicates that the efficiency acquired during evolution persists on held-out trajectories without any test-time modification. Runtime follows the same trend, with ECDYSIS (w/ FDCR) reducing the mean from 126.08 to 118.69 seconds per trajectory. The largest gain is on Qwen3-14B over τ^2 -Airline, where the mean runtime drops from 197.61 to 57.16 seconds, corresponding to a 3.46 $\times$ speedup. Because runtime also reflects model service latency and trajectory length, we report the full means and standard deviations in Table 3 rather than only the aggregate.

Table 2: Held-out task performance across five LLMs and two datasets.

Model	Dataset	Method	Accuracy (%)		
			AVG	Pass@3	Pass^3
Qwen3-8B	τ^2 -Airline	Direct	16.67 $\pm$ 7.64	35.00	0.00
		Human-Aug.	55.00 $\pm$ 5.00	65.00	40.00
		Self-Evolution	35.00 $\pm$ 10.00	50.00	20.00
		ECDYSIS (w/o FDCR)	48.33 $\pm$ 5.77	55.00	45.00
		ECDYSIS (w/ FDCR)	60.00 $\pm$ 5.00	80.00	40.00
	τ^2 -Retail	Direct	41.67 $\pm$ 5.77	70.00	15.00
		Human-Aug.	48.33 $\pm$ 2.89	80.00	20.00
		Self-Evolution	41.67 $\pm$ 15.28	75.00	15.00
		ECDYSIS (w/o FDCR)	53.33 $\pm$ 2.89	75.00	25.00
		ECDYSIS (w/ FDCR)	63.33 $\pm$ 7.64	85.00	45.00
Qwen3-14B	τ^2 -Airline	Direct	16.67 $\pm$ 7.64	30.00	0.00
		Human-Aug.	38.33 $\pm$ 10.41	55.00	25.00
		Self-Evolution	23.33 $\pm$ 7.64	40.00	10.00
		ECDYSIS (w/o FDCR)	41.67 $\pm$ 7.64	60.00	25.00
		ECDYSIS (w/ FDCR)	43.33 $\pm$ 7.64	65.00	25.00
	τ^2 -Retail	Direct	25.00 $\pm$ 5.00	50.00	5.00
		Human-Aug.	45.00 $\pm$ 13.23	65.00	20.00
		Self-Evolution	43.33 $\pm$ 12.58	65.00	20.00
		ECDYSIS (w/o FDCR)	48.33 $\pm$ 5.77	70.00	35.00
		ECDYSIS (w/ FDCR)	53.33 $\pm$ 2.89	70.00	30.00
Qwen3-32B	τ^2 -Airline	Direct	20.00 $\pm$ 5.00	35.00	10.00
		Human-Aug.	56.67 $\pm$ 14.43	70.00	40.00
		Self-Evolution	51.67 $\pm$ 11.55	80.00	20.00
		ECDYSIS (w/o FDCR)	61.67 $\pm$ 5.77	80.00	40.00
		ECDYSIS (w/ FDCR)	68.33 $\pm$ 2.89	80.00	50.00
	τ^2 -Retail	Direct	58.33 $\pm$ 2.89	75.00	35.00
		Human-Aug.	58.33 $\pm$ 7.64	80.00	45.00
		Self-Evolution	65.00 $\pm$ 5.00	75.00	55.00
		ECDYSIS (w/o FDCR)	70.00 $\pm$ 8.66	85.00	45.00
		ECDYSIS (w/ FDCR)	75.00 $\pm$ 0.00	85.00	60.00
MiniMax-M2.7	τ^2 -Airline	Direct	80.00 $\pm$ 5.00	95.00	60.00
		Human-Aug.	81.67 $\pm$ 5.77	90.00	70.00
		Self-Evolution	73.33 $\pm$ 2.89	90.00	45.00
		ECDYSIS (w/o FDCR)	81.67 $\pm$ 2.89	85.00	80.00
		ECDYSIS (w/ FDCR)	86.67 $\pm$ 7.64	95.00	70.00
	τ^2 -Retail	Direct	90.00 $\pm$ 5.00	100.00	80.00
		Human-Aug.	93.33 $\pm$ 5.77	100.00	85.00
		Self-Evolution	95.00 $\pm$ 5.00	100.00	90.00
		ECDYSIS (w/o FDCR)	100.00 $\pm$ 0.00	100.00	100.00
		ECDYSIS (w/ FDCR)	96.67 $\pm$ 2.89	100.00	90.00
Llama-3.1-8B	τ^2 -Airline	Direct	25.00 $\pm$ 8.66	35.00	15.00
		Human-Aug.	31.67 $\pm$ 7.64	40.00	20.00
		Self-Evolution	28.33 $\pm$ 10.41	45.00	10.00
		ECDYSIS (w/o FDCR)	31.67 $\pm$ 5.77	40.00	20.00
		ECDYSIS (w/ FDCR)	35.00 $\pm$ 5.00	40.00	30.00
	τ^2 -Retail	Direct	8.33 $\pm$ 2.89	10.00	5.00
		Human-Aug.	8.33 $\pm$ 2.89	15.00	5.00
		Self-Evolution	10.00 $\pm$ 0.00	15.00	5.00
		ECDYSIS (w/o FDCR)	10.00 $\pm$ 5.00	15.00	5.00
		ECDYSIS (w/ FDCR)	11.67 $\pm$ 2.89	15.00	10.00

7 Analysis of Evolution Process

To further explain the training efficiency results in Section 5.3, this section analyzes the actual execution of different harness evolution methods from four aspects: candidate generation, API cost, token usage, and end-to-end training time. All statistics are collected from the complete evolution runs described in Section 4.1. The three methods use the same training evaluation, candidate validation, rollback, and early stopping protocols. We report the model calls and resource usage observed

Table 3: Inference cost results across LLMs, datasets, and methods. Inference efficiency is measured by token consumption (M) and execution time (s).

Model	Dataset	Method	Efficiency	
			Tokens (M)	Time (s)
Qwen3-8B	τ^2 -Airline	Direct	9.544 ± 0.352	67.55 ± 3.07
		Human-Aug.	13.961 ± 0.666	108.58 ± 15.42
		Self-Evolution	12.321 ± 1.205	175.23 ± 27.82
		ECDYSIS (w/o FDCR)	12.516 ± 0.428	102.43 ± 1.59
		ECDYSIS (w/ FDCR)	12.070 ± 1.458	98.87 ± 17.85
	τ^2 -Retail	Direct	8.470 ± 0.416	108.92 ± 15.41
		Human-Aug.	9.536 ± 0.182	113.88 ± 2.25
		Self-Evolution	9.483 ± 0.495	171.32 ± 18.96
		ECDYSIS (w/o FDCR)	8.205 ± 0.118	180.13 ± 20.16
		ECDYSIS (w/ FDCR)	9.790 ± 0.326	142.30 ± 20.28
Qwen3-14B	τ^2 -Airline	Direct	13.685 ± 0.858	103.27 ± 1.99
		Human-Aug.	15.718 ± 1.539	101.83 ± 19.15
		Self-Evolution	24.308 ± 0.420	197.61 ± 11.94
		ECDYSIS (w/o FDCR)	16.089 ± 1.954	157.02 ± 31.85
		ECDYSIS (w/ FDCR)	16.417 ± 1.462	57.16 ± 11.15
	τ^2 -Retail	Direct	19.239 ± 2.460	134.06 ± 20.80
		Human-Aug.	20.099 ± 3.226	158.45 ± 42.92
		Self-Evolution	14.392 ± 3.268	107.25 ± 31.44
		ECDYSIS (w/o FDCR)	18.424 ± 2.998	128.51 ± 49.22
		ECDYSIS (w/ FDCR)	12.746 ± 1.282	90.82 ± 18.23
Qwen3-32B	τ^2 -Airline	Direct	6.469 ± 0.349	75.51 ± 9.04
		Human-Aug.	11.047 ± 0.287	105.05 ± 8.83
		Self-Evolution	13.206 ± 0.483	117.99 ± 3.33
		ECDYSIS (w/o FDCR)	9.748 ± 0.578	91.34 ± 9.55
		ECDYSIS (w/ FDCR)	11.670 ± 1.009	100.36 ± 13.24
	τ^2 -Retail	Direct	7.182 ± 0.066	77.40 ± 4.93
		Human-Aug.	12.297 ± 1.813	88.55 ± 11.70
		Self-Evolution	9.023 ± 0.353	79.43 ± 14.13
		ECDYSIS (w/o FDCR)	7.895 ± 0.220	73.45 ± 1.11
		ECDYSIS (w/ FDCR)	9.015 ± 0.299	102.02 ± 5.28
MiniMax-M2.7	τ^2 -Airline	Direct	5.252 ± 0.171	135.52 ± 12.59
		Human-Aug.	6.554 ± 0.074	154.62 ± 3.26
		Self-Evolution	8.161 ± 0.327	170.69 ± 15.05
		ECDYSIS (w/o FDCR)	6.127 ± 0.083	139.25 ± 3.80
		ECDYSIS (w/ FDCR)	6.526 ± 0.218	202.03 ± 9.94
	τ^2 -Retail	Direct	5.173 ± 0.048	115.32 ± 4.57
		Human-Aug.	6.054 ± 0.077	270.38 ± 40.65
		Self-Evolution	5.728 ± 0.052	164.34 ± 3.70
		ECDYSIS (w/o FDCR)	5.779 ± 0.119	335.37 ± 43.90
		ECDYSIS (w/ FDCR)	5.804 ± 0.051	201.45 ± 4.04
Llama-3.1-8B	τ^2 -Airline	Direct	5.818 ± 0.360	32.23 ± 5.01
		Human-Aug.	9.640 ± 0.144	51.68 ± 3.22
		Self-Evolution	10.319 ± 0.299	43.38 ± 1.74
		ECDYSIS (w/o FDCR)	10.321 ± 0.434	51.97 ± 4.52
		ECDYSIS (w/ FDCR)	9.015 ± 0.178	159.60 ± 15.38
	τ^2 -Retail	Direct	6.181 ± 0.266	29.31 ± 4.14
		Human-Aug.	8.586 ± 0.450	32.32 ± 5.33
		Self-Evolution	8.724 ± 0.276	33.53 ± 5.73
		ECDYSIS (w/o FDCR)	8.438 ± 0.254	54.72 ± 1.40
		ECDYSIS (w/ FDCR)	8.517 ± 0.228	32.30 ± 4.29

during actual execution. Therefore, these results reflect the cost of different candidate generation mechanisms along their actual execution paths rather than theoretical budgets normalized by a fixed number of evolution rounds.

7.1 Candidate Generation

Different evolution methods exhibit different candidate generation costs. Compared with SE, ECDYSIS requires fewer coding-agent calls, mainly because the two methods use different update granulari-

Table 4: Comparison of API costs between ECDYSIS and baseline methods during harness training.

Dataset	Method	Training Evaluation	Evolution	Total
τ^2 -Airline	Self-Evolution	\$0.837	\$5.545	\$6.382
	ECDYSIS (w/o FDCR)	\$0.967	\$1.169	\$2.136
	ECDYSIS (w/ FDCR)	\$0.870	\$1.739	\$2.609
τ^2 -Retail	Self-Evolution	\$0.609	\$7.875	\$8.484
	ECDYSIS (w/o FDCR)	\$0.555	\$1.930	\$2.485
	ECDYSIS (w/ FDCR)	\$0.506	\$5.257	\$5.763

Table 5: Input-cache utilization during harness evolution.

Dataset	Method	Cached Tokens	Total Input Tokens	Cache Hit Rate (%)
τ^2 -Retail	Self-Evolution	16,375,808	18,327,058	89.35
	ECDYSIS (w/o FDCR)	7,570,944	7,885,487	96.01
	ECDYSIS (w/ FDCR)	19,547,136	20,485,995	95.42
τ^2 -Airline	Self-Evolution	12,368,896	13,824,385	89.47
	ECDYSIS (w/o FDCR)	4,384,768	4,601,409	95.29
	ECDYSIS (w/ FDCR)	5,103,104	5,470,490	93.28

ties. SE generates and accumulates modifications for individual failures, while ECDYSIS aggregates the failure evidence within each evolution round and generates candidate modifications from the aggregated evidence. As the number of failures within a round increases, the candidate generation cost of SE grows accordingly, while ECDYSIS continues to generate candidates at the round level. The runtime logs further reveal differences between the two methods during actual candidate generation. For τ^2 -Retail, approximately 67% of the coding-agent calls from SE completed normally, compared with approximately 97% in τ^2 -Airline. In contrast, all candidate generation calls from ECDYSIS completed normally in both domains. These results show that accumulating modifications for individual failures not only increases candidate generation cost but is also associated with a higher proportion of incomplete calls in some runs. By aggregating failure evidence, ECDYSIS reduces repeated candidate generation and exhibits more stable execution behavior in our experiments.

7.2 API Cost and Token Usage

We divide the API cost of the complete training process into training evaluation and evolution (Table 4). Training evaluation covers model calls made during training task execution. Evolution covers the cost of candidate analysis and implementation after failure evidence is generated, including calls to the coding agent, FDCR when enabled, and other model calls during the evolution process. Each cost is calculated from the input tokens, output tokens, and API prices applicable at execution time, as recorded in the call logs. The training evaluation cost remains below \$1 for all three methods, with most cost differences arising during evolution. SE incurs total costs of \$8.484 and \$6.382, respectively. ECDYSIS (w/o FDCR) reduces these costs to \$2.485 and \$2.136, corresponding to reductions of 70.71% and 66.53%. The total costs of ECDYSIS (w/ FDCR) are \$5.763 and \$2.609, representing reductions of 32.07% and 59.12% relative to SE. Overall, both ECDYSIS configurations incur lower total API costs than SE. This result is consistent with the reduction in coding-agent calls discussed above and suggests that generating candidates from failure evidence aggregated over an entire round can effectively reduce the cost of repeated implementation. SE achieves cache hit rates of 89.35% and 89.47% on τ^2 -Retail and τ^2 -Airline, respectively (see Table 5). In comparison, the cache hit rates of ECDYSIS (w/o FDCR) increase to 96.01% and 95.29%, while ECDYSIS (w/ FDCR) reaches 95.42% and 93.28%. Notably, on τ^2 -Retail, ECDYSIS (w/ FDCR) incurs substantially lower total API cost than SE despite using more input tokens. These results further demonstrate the cost efficiency of ECDYSIS.

7.3 End-to-End Evolution Time

We report the end-to-end training time from the initial training evaluation to the point when the final harness passes validation and is frozen in Table 6. For τ^2 -Retail, ECDYSIS (w/o FDCR) and ECDYSIS (w/ FDCR) achieve end-to-end training speedups of $1.42\times$ and $1.30\times$ over SE, respectively.

Table 6: Training time (s) for the three harness self-evolution methods.

Dataset	SE	ECDYSIS (w/o FDCR)	ECDYSIS (w/ FDCR)	Speedup
τ^2 -Retail	1,831.4	1,292.4	1,405.9	$1.42\times / 1.30\times$
τ^2 -Airline	8,120.6	2,510.8	4,403.0	$3.23\times / 1.84\times$

For τ^2 -Airline, the corresponding speedups reach $3.23\times$ and $1.84\times$. These results indicate that generating candidates from failure evidence aggregated over an entire round reduces redundant work during evolution and consistently shortens end-to-end training time across both domains.

8 Task Structure Analysis

Motivation. Recent work on on-policy distillation (OPD) shows that training efficiency depends not only on the amount of training data, but also on the information contained in individual training instances. Hou et al. [32] find that a small set of carefully selected hard examples can nearly match training on a much larger dataset, with the gains attributed primarily to the longer reasoning trajectories induced by challenging problems. This raises a related question for harness evolution: *which training instances provide the most informative signal for harness training?* Unlike model training, where longer reasoning trajectories can be particularly valuable, harness training requires sufficient task complexity and behavioral breadth to exercise diverse interaction structures and expose distinct failure mechanisms. We therefore view training tasks not merely as execution instances, but as diagnostic probes that reveal both the coverage of runtime behaviors and the deficiencies of the harness.

8.1 Cross-Task Structural Regularities

Although Retail and Airline tasks differ in entities, tools, and user goals, they exhibit recurring interaction structures. Multi-step and multi-intent tasks require persistent goal tracking; conditional branches require adaptation to intermediate tool results; state constraints impose preconditions on subsequent actions; and cross-tool dependencies require consistent identification and propagation of entities, attributes, and relations. High-risk operations, intent changes, and interruptions further require appropriate confirmation, termination, or plan revision. These recurring structures give rise to common harness failure modes across tasks with different surface semantics. For example, stale information after state updates, ambiguous entity references, and downstream tool calls without required prerequisite information reflect deficiencies in state synchronization, entity resolution, and tool coordination, respectively. Similarly, failures involving state checks, entity propagation, operation preconditions, and tool ordering can recur across training and held-out tasks despite substantial differences in their surface forms.

This observation suggests that effective harness improvements should capture underlying runtime constraints rather than memorize task-specific failures. For example, the harness should re-query state after relevant updates, resolve ambiguous entities before side-effecting operations, and verify required dependency information before invoking downstream tools. Such abstractions allow improvements identified from one task to generalize to structurally related failures in unseen tasks.

8.2 Insights for Training Data Curation

The cross-task regularities motivate a failure-aware approach to training data curation. Training tasks differ in the diagnostic evidence they provide: some expose novel interaction structures or previously uncovered harness deficiencies, whereas others produce redundant manifestations of known failure mechanisms. Consequently, selecting training data based solely on task quantity or surface diversity can incur substantial execution cost without providing commensurate information for harness evolution. We therefore propose a failure-aware training data selection strategy that jointly considers interaction structures and failure mechanisms across task instances. Specifically, we prioritize tasks that expose novel execution paths or previously under-covered harness deficiencies, while reducing tasks that provide redundant failure signals. This yields a more informative training set with broader coverage of runtime behaviors and fewer redundant executions. Data curation is closely coupled with failure diagnosis. Failures observed on individual tasks may represent different

surface manifestations of the same underlying harness deficiency. By aggregating failure evidence across task instances, ECDYSIS identifies shared deficiencies and generates more generalizable harness modifications, rather than repeatedly addressing isolated failures. Overall, effective harness training should prioritize informative tasks over data quantity. By selecting tasks with novel execution paths and failure mechanisms, ECDYSIS enables more efficient harness evolution with fewer, more diagnostic training tasks.

To further validate the potential of training data curation, we conduct an additional experiment using five training failures, corresponding to one-quarter of the original training set. We train ECDYSIS with this reduced training set and compare the resulting performance against that of full-data training. The results are summarized in Table 13. For a fair comparison and ablation study, we also evaluate the training results obtained by randomly selecting five training samples. Overall, reduced-data training achieves performance comparable to full-data training while substantially reducing training cost during harness evolution. In contrast, the random selection strategy yielded inferior performance.

8.3 Beyond Local Failure-Driven Evolution

To better understand how failure-driven evolution modifies the runtime harness, we conducted a fine-grained manual analysis of the training-time data and quantified t introduced in Section 3.1. Here, t denotes the proportion of coding-agent-issued modification decisions that accommodate limitations of the task model, among all independent modification decisions. We find that local failure-driven evolution can overfit to task-specific outcomes by promoting a particular training-task answer into a general runtime constraint. In some cases, the evolution agent removes a legitimate action option to avoid a specific model error, replacing a conditional decision that should be made by the model with a global harness-level prohibition. While such modifications may improve the triggering evaluation case, they also shrink the valid action space of the harness and may impair its generalization across models.

Quantitatively, we observe $t = 60.0\%$ for Self-Evolution (SE), compared with $t = 45.5\%$ for ECDYSIS, corresponding to a reduction of 14.5 percentage points. This result suggests that local failure-driven evolution is more prone to adapting the harness to model-specific deficiencies, whereas the cross-task failure analysis in ECDYSIS provides broader evidence before committing to a harness-level modification. Conceptually, the objective of ECDYSIS can be viewed as attenuating the model-accommodation component from t to $t' = \beta t$, where $\beta \in (0, 1)$, rather than treating every observed failure as direct evidence for modifying the harness. By reducing such model-specific over-adaptation, ECDYSIS can preserve a more general valid action space and reduce the dependence of the resulting harness on the model used during evolution, thereby improving its cross-model generalization.

9 Conclusion

In this paper, we present ECDYSIS, an efficient framework for self-evolving runtime harnesses for LLM agents. Our central insight is that execution failures are not uniformly actionable: they may reflect model-specific limitations rather than harness deficiencies. Treating individual failures as direct modification signals can therefore over-accommodate the evolving model and compromise harness generalization. ECDYSIS addresses this challenge by shifting failure-driven evolution from individual failures to recurring cross-task failure patterns. By aggregating failure evidence across instances and applying FDCR, it provides stronger evidence for systematic harness deficiencies before committing to harness-level modifications. Across multiple LLMs and reasoning benchmarks, ECDYSIS improves harness training efficiency and agent reasoning performance while achieving stronger cross-model generalization and lower token consumption. Fine-grained analysis further reveals that reducing model-specific accommodation is closely associated with more generalizable harness evolution. Overall, our findings suggest that effective harness evolution requires not only correcting failures, but also determining which failures constitute reliable evidence for changes that generalize beyond the model and tasks used during evolution.

Ethical Considerations

In this paper, AI assistants are used to polish the writing. We also use AI agents to assist with programming. All AI-assisted outputs are reviewed and verified by the authors.

References

- [1] Zhaoling Chen, Robert Tang, Gangda Deng, Fang Wu, Jialong Wu, Zhiwei Jiang, Viktor Prasanna, Arman Cohan, and Xingyao Wang. LocAgent: Graph-guided LLM agents for code localization. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8697–8727, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.426. URL <https://aclanthology.org/2025.acl-long.426/>.
- [2] Che Jiang, Jincheng Zhong, Yu Fu, Kai Tian, Junlin Yang, Kaikai Zhao, Yuchong Wang, Tianwei Luo, Weizhi Wang, Yuxin Zuo, et al. Self-improving agents in the era of experience: A survey of self-to meta-evolution. 2026.
- [3] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.
- [4] Guangya Wan, Mingyang Ling, Xiaoqi Ren, Rujun Han, Sheng Li, and Zizhao Zhang. COMPASS: Enhancing agent long-horizon reasoning with evolving context. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens, editors, *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3360–3380, San Diego, California, United States, July 2026. Association for Computational Linguistics. ISBN 979-8-89176-390-6. doi: 10.18653/v1/2026.acl-long.152. URL <https://aclanthology.org/2026.acl-long.152/>.
- [5] Ruixin Zhang, Wuyang Dai, Hung Viet Pham, Gias Uddin, Jinqiu Yang, and Song Wang. *Engineering Pitfalls in AI Coding Tools: An Empirical Study of Bugs in Claude Code, Codex, and Gemini CLI*, page 393–403. Association for Computing Machinery, New York, NY, USA, 2026. ISBN 9798400726361. URL <https://doi.org/10.1145/3803437.3805213>.
- [6] Hyunin Lee, Jinglue Xu, Jeffrey Seely, Donghyun Lee, Matei Zaharia, and Yujin Tang. Recursive harness self-improvement, 2026. URL <https://arxiv.org/abs/2607.15524>.
- [7] Mengzhuo Chen, Junjie Wang, Zhe Liu, Yawen Wang, Haiming Zheng, and Qing Wang. From failed trajectories to reliable llm agents: Diagnosing and repairing harness flaws. *arXiv preprint arXiv:2606.06324*, 2026.
- [8] Wenyi Wang, Piotr Piękos, Li Nanbo, Firas Laakom, Yimeng Chen, Mateusz Ostaszewski, Mingchen Zhuge, and Jürgen Schmidhuber. Huxley-g\`odel machine: Human-level coding agent development by an approximation of the optimal self-improving machine. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=T0EiEuh0OL>.
- [9] Shuai Shao, Kangning Zhang, Qingyao Li, Shijian Wang, Hao Wang, Wenxiang Jiao, Yuan Lu, Yi Guo, Weiwen Liu, and Weinan Zhang. Harness-r1: Learning to edit executable runtime harnesses from agent failure trajectories. *arXiv preprint arXiv:2608.02276*, 2026.
- [10] Shuai Shao, Qihan Ren, Dongrui Liu, Chen Qian, Boyi Wei, Dadi Guo, Yang JingYi, Xinhao Song, Linfeng Zhang, Weinan Zhang, and Jing Shao. Your agent may misevolve: Emergent risks in self-evolving LLM agents. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=FdljgQQW28>.
- [11] Wen Jiang, Mingmin Chu, Yimeng Tian, Qianxin Zhang, Haofei Yang, Rui Yang, Yang Liu, Tao Lv, and Fangming Li. Harnessesolve: Learning from reference trajectories for reliable agent self-evolution. *arXiv preprint arXiv:2609.00829*, 2026.
- [12] Yike Wang, Huaisheng Zhu, Zhengyu Hu, Yige Yuan, Zhengyu Chen, Shakti Senthil, Hannaneh Hajishirzi, Yulia Tsvetkov, Pradeep Dasigi, and Teng Xiao. Rethinking the evaluation of harness evolution for agents. In *COLM 2026 The 2nd Workshop on Lifelong Agents: Learning, Aligning, and Evolving*, 2026. URL <https://openreview.net/forum?id=WVAeeSlVim>.

- [13] Peiying Yu, Guoxin Chen, and Jingjing Wang. Table-critic: A multi-agent framework for collaborative criticism and refinement in table reasoning. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 17432–17451, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.853. URL <https://aclanthology.org/2025.acl-long.853/>.
- [14] Justin Chen, Archiki Prasad, Swarnadeep Saha, Elias Stengel-Eskin, and Mohit Bansal. MAgICoRe: Multi-agent, iterative, coarse-to-fine refinement for reasoning. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 32663–32686, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi: 10.18653/v1/2025.emnlp-main.1660. URL <https://aclanthology.org/2025.emnlp-main.1660/>.
- [15] Jingsen Zhang, Zihang Tian, Xueyang Feng, Xu Chen, and Chong Chen. Enhancing recommendation explanations through user-centric refinement. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 8177–8191, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-335-7. doi: 10.18653/v1/2025.findings-emnlp.434. URL <https://aclanthology.org/2025.findings-emnlp.434/>.
- [16] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: language agents with verbal reinforcement learning. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 8634–8652. Curran Associates, Inc., 2023. doi: 10.52202/075280-0377. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/1b44b878bb782e6954cd888628510e90-Paper-Conference.pdf.
- [17] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan A, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into state-of-the-art pipelines. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=sY5N0zY5Od>.
- [18] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, and Y. Sun, editors, *International Conference on Learning Representations*, volume 2024, pages 12028–12068, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/file/3339f19c5fcee3ad74502947a32be9e6-Paper-Conference.pdf.
- [19] Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. In Y. Yue, A. Garg, N. Peng, F. Sha, and R. Yu, editors, *International Conference on Learning Representations*, volume 2025, pages 21344–21377, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/file/36b7acf6f6010652b3f2a433774a66fe-Paper-Conference.pdf.
- [20] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, XiongHui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. Aflow: Automating agentic workflow generation. In Y. Yue, A. Garg, N. Peng, F. Sha, and R. Yu, editors, *International Conference on Learning Representations*, volume 2025, pages 34040–34077, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/file/5492ecbce4439401798dcd2c90be94cd-Paper-Conference.pdf.
- [21] Jenny Zhang, Shengran Hu, Cong Lu, Robert Tjarko Lange, and Jeff Clune. Darwin gödel machine: Open-ended evolution of self-improving agents. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=pUpzQZTvGY>.

- [22] Hangfan Zhang, Shao Zhang, Kangcong Li, Chen Zhang, Yang Chen, Yiqun Zhang, Lei Bai, and Shuyue Hu. Self-harness: Harnesses that improve themselves, 2026. URL <https://arxiv.org/abs/2606.09498>.
- [23] Xiao Yu, Baolin Peng, Ruize Xu, Hao Zou, Qianhui Wu, Hao Cheng, Wenlin Yao, Nikhil Singh, Zhou Yu, and Jianfeng Gao. Openforgerl: Train harness-native agents in any environment, 2026. URL <https://arxiv.org/abs/2607.21557>.
- [24] Ruhan Wang, Yucheng Shi, Zongxia Li, Zhongzhi Li, Yue Yu, Junyao Yang, Kishan Panaganti, Haitao Mi, Dongruo Zhou, and Leoweiliang. Harness handbook: Making evolving agent harnesses readable, navigable, and editable, 2026. URL <https://arxiv.org/abs/2607.13285>.
- [25] Yue Huang, Wenjie Wang, Han Bao, Yuchen Ma, Xiaonan Luo, Yi Nian, Haomin Zhuang, Zheyuan Liu, Yue Zhao, and Xiangliang Zhang. Memoharness: Agent harnesses that learn from experience, 2026. URL <https://arxiv.org/abs/2607.14159>.
- [26] Jinghan Xu, Yikai Zhang, Aili Chen, Weiyan Li, Jiaqing Liang, and Deqing Yang. Verify smarter, evolve further: Efficient harness evolution through behavior-aware verification. *arXiv preprint arXiv:2608.27311*, 2026.
- [27] Tianshi Xu, Huifeng Wen, and Meng Li. Adapting the interface, not the model: Runtime harness adaptation for deterministic llm agents. *arXiv preprint arXiv:2605.22166*, 2026.
- [28] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yingrui Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.
- [29] Anyi Xu, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, Chenchen Ling, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chengyu Hou, Chenhao Xu, Chenze Shao, Chong Ruan, Conner Sun, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Donghao Li, Dongjie Ji, Erhang Li, Fang Wei, Fangyun Lin, Fangzhou Yuan, Feiyu Xia, Fucong Dai, Guangbo Hao, Guanting Chen, Guoai Cao, Guolai Meng, Guowei Li, Han Yu, Han Zhang, Hanwei Xu, Hao Li, Haofen Liang, Haoling Zhang, Haoming Luo, Haoran Wei, Haotian Yuan, Haowei Zhang, Haowen Luo, Haoyu Chen, Haozhe Ji, Hengqing Zhang, Honghui Ding, Hongxuan Tang, Huanqi Cao, Huazuo Gao, Hui Qu, Hui Zeng, J Yang, JQ Zhu, Jia Luo, Jia Song, Jia Yu, Jialiang Huang, Jialu Cai, Jian Liang, Jiangting Zhou, Jiasheng Ye, Jiashi Li, Jiaxin Xu, Jiewen Hu, Jieyu Yang, Jin Chen, Jin Yan, Jingchang Chen, Jingli Zhou, Jingting Xiang, Jingyang Yuan, Jingyuan Cheng, Jingzi Zhou, Jinhua Zhu, Jiping Yu, Joseph Sun, Jun Ran, Junguang Jiang, Junjie Qiu, Junlong Li, Junmin Zheng, Junxiao Song, Kai Dong, Kaige Gao, Kang Guan, Kexing Zhou, Kezhao Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Wang, Leyi Xia, Li Zhang, Liang Zhao, Lihua Guo, Lingxiao Luo, Linwang Ma, Linyan Zhu, Litong Wang, Liyu Cai, Liyue Zhang, Longhao Chen, MS Di, MY Xu, Max Mei, Miaojun Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Mingxu Zhou, Minmin Han, Ning Wang, Panpan Huang, Panpan Wang, Peixin Cong, Peiyi Wang, Peng Zhang, Qiancheng Wang, Qihao Zhu, Qingyang Li, Qinyu Chen, Qiushi Du, Qiwei Jiang, Rui Tian, Ruifan Xu, Ruijie Lu, Ruiling Xu, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runqian Chen, Runqiu Yin, Runxin Xu, Ruomeng Shen, Ruoyu Zhang, Ruyi Chen, SH Liu, Shanghao Lu, Shangmian Sun, Shangyan Zhou, Shanhuang Chen, Shaofei Cai, Shaoheng Nie, Shaoqing Wu, Shaoyuan Chen, Shengding Hu, Shengyu Liu, Shiqiang Hu, Shirong Ma, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, Shuying Yu, Songyang Zhou, Tao Ni, Tao Yun, Tian Jin, Tian Pei, Tian Ye, Tianle Lin, Tianran Ji, Tianyi Cui, Tianyuan Yue, Tingting Yu, Tun Wang, W Zhang, WL Xiao, Wangding Zeng, Wei An, Weilin Zhao, Wen Liu, Wenfeng Liang, Wenjie Pang, Wenjing Luo, Wenjing Yao, Wenjun Gao, Wenkai Yang, Wenlv Huang, Wenqing Hou, Wentao Zhang, Wenting Ma, Xi Gao, Xiang He, Xiangwen Wang, Xianzu

- Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaokang Zhang, Xiaotao Nie, Xiaowen Sun, Xiaoxiang Wang, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingchen Liu, Xingkai Yu, Xingyou Li, Xinyu Yang, Xinyu Zhang, Xu Chen, Xuanyu Wang, Xuecheng Su, Xueyin Chen, Xuheng Lin, Xuwei Fu, YC Yan, YQ Wang, YW Ma, Yanfeng Luo, Yang Zhang, Yanhong Xu, Yanru Ma, Yanwen Huang, Yao Li, Yao Li, Yao Xu, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Qian, Yi Shao, Yi Yu, Yichao Zhang, Yifan Ding, Yifan Shi, Yijia Wu, Yiliang Xiong, Yiling Ma, Ying He, Ying Tang, Ying Zhou, Yingjia Luo, Yinmin Zhong, Yishi Piao, Yisong Wang, Yixiang Zhang, Yixiao Chen, Yixuan Tan, Yixuan Wei, Yiyang Ma, Yiyuan Liu, Yonglun Yang, Yongqiang Guo, Yongtong Wu, Yu Wu, YuKun Li, Yuan Cheng, Yuan Ou, Yuanfan Xu, Yuanhao Li, Yuduan Wang, Yuehan Yang, Yuer Xu, Yuhan Wu, Yuhao Meng, Yuheng Zou, Yukun Zha, Yunfan Xiong, Yupeng Chen, Yuping Lin, Yuqian Cao, Yuqian Wang, Yushun Zhang, Yuting Yan, Yutong Lin, Yuxian Gu, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuxuan Zhou, Yuyang Zhou, Yuzhen Huang, ZF Wu, Zehao Wang, Zehua Zhao, Zehui Ren, Zekai Zhang, Zhangli Sha, Zhe Fu, Zhe Ju, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zheren Gao, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhixian Huang, Zhixuan Chen, Zhiyu Wu, Zhizhou Ren, Zhongyu Wu, Zhuoshu Li, Zhuping Zhang, Zian Xu, Zihao Wang, Zihua Qu, Zihui Gu, Zijia Zhu, Zilin Li, Zipeng Zhang, Ziwei Xie, Ziyi Gao, Ziyi Wan, Zizheng Pan, and Zongqing Yao. Deepseek-v4: Towards highly efficient million-token context intelligence, 2026. URL <https://arxiv.org/abs/2606.19348>.
- [30] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. Agentbench: Evaluating LLMs as agents. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=zAdUB0aCTQ>.
- [31] Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik R Narasimhan. τ^2 -bench: Evaluating conversational agents in a dual-control environment. In *Forty-third International Conference on Machine Learning*, 2026. URL <https://openreview.net/forum?id=OC2z7iSQKa>.
- [32] Zhinan Hou, Jiaqi Zhang, Xunliang Cai, and Keyou You. What matters in on-policy distillation? a perspective on data efficiency and data selection, 2026. URL <https://arxiv.org/abs/2609.05198>.

Table 7: Accuracy results across five models on AgentBench.

Model	Method	Accuracy (%)		
		AVG	Pass@3	Pass^3
Qwen3-8B	Direct	10.00	15.00	5.00
	Human-Aug.	85.00	90.00	80.00
	Self-Evolution	90.00	90.00	90.00
	ECDYSIS (w/o FDCR)	85.00	85.00	85.00
	ECDYSIS (w/ FDCR)	90.00	90.00	90.00
Qwen3-14B	Direct	28.33	30.00	25.00
	Human-Aug.	80.00	80.00	80.00
	Self-Evolution	85.00	85.00	85.00
	ECDYSIS (w/o FDCR)	85.00	85.00	85.00
	ECDYSIS (w/ FDCR)	90.00	90.00	90.00
Qwen3-32B	Direct	40.00	45.00	30.00
	Human-Aug.	75.00	75.00	75.00
	Self-Evolution	85.00	85.00	85.00
	ECDYSIS (w/o FDCR)	85.00	85.00	85.00
	ECDYSIS (w/ FDCR)	90.00	90.00	90.00
MiniMax-M2.7	Direct	80.00	90.00	65.00
	Human-Aug.	78.33	85.00	70.00
	Self-Evolution	83.33	85.00	80.00
	ECDYSIS (w/o FDCR)	86.67	90.00	85.00
	ECDYSIS (w/ FDCR)	90.00	90.00	90.00
Llama-3.1-8B	Direct	5.00	5.00	5.00
	Human-Aug.	68.33	70.00	65.00
	Self-Evolution	70.00	70.00	70.00
	ECDYSIS (w/o FDCR)	80.00	80.00	80.00
	ECDYSIS (w/ FDCR)	90.00	90.00	90.00

Table 8: Inference cost results across five LLMs and methods on AgentBench.

Model	Method	Efficiency	
		Tokens	Time
Qwen3-8B	Direct	5.615 $\pm$ 0.073	96.09 $\pm$ 3.63
	Human-Aug.	1.562 $\pm$ 0.058	46.67 $\pm$ 0.73
	Self-Evolution	1.418 $\pm$ 0.094	48.21 $\pm$ 1.70
	ECDYSIS (w/o FDCR)	1.413 $\pm$ 0.001	47.24 $\pm$ 0.70
	ECDYSIS (w/ FDCR)	1.413 $\pm$ 0.028	45.68 $\pm$ 0.74
Qwen3-14B	Direct	4.853 $\pm$ 0.089	198.25 $\pm$ 1.10
	Human-Aug.	1.645 $\pm$ 0.000	52.01 $\pm$ 0.06
	Self-Evolution	1.618 $\pm$ 0.001	56.07 $\pm$ 0.34
	ECDYSIS (w/o FDCR)	1.392 $\pm$ 0.002	51.77 $\pm$ 1.12
	ECDYSIS (w/ FDCR)	1.514 $\pm$ 0.003	54.32 $\pm$ 2.20
Qwen3-32B	Direct	4.330 $\pm$ 0.183	98.18 $\pm$ 4.22
	Human-Aug.	2.139 $\pm$ 0.007	61.74 $\pm$ 0.26
	Self-Evolution	1.789 $\pm$ 0.026	60.79 $\pm$ 1.39
	ECDYSIS (w/o FDCR)	1.436 $\pm$ 0.001	54.40 $\pm$ 1.12
	ECDYSIS (w/ FDCR)	1.330 $\pm$ 0.006	51.54 $\pm$ 0.83
MiniMax-M2.7	Direct	2.937 $\pm$ 0.444	150.48 $\pm$ 18.61
	Human-Aug.	2.310 $\pm$ 0.119	162.26 $\pm$ 10.01
	Self-Evolution	2.066 $\pm$ 0.065	169.65 $\pm$ 6.06
	ECDYSIS (w/o FDCR)	1.664 $\pm$ 0.118	114.02 $\pm$ 4.39
	ECDYSIS (w/ FDCR)	1.474 $\pm$ 0.025	90.57 $\pm$ 3.43
Llama-3.1-8B	Direct	4.771 $\pm$ 0.170	368.80 $\pm$ 13.20
	Human-Aug.	2.076 $\pm$ 0.135	161.42 $\pm$ 7.92
	Self-Evolution	1.952 $\pm$ 0.197	71.34 $\pm$ 3.51
	ECDYSIS (w/o FDCR)	1.225 $\pm$ 0.000	44.43 $\pm$ 0.48
	ECDYSIS (w/ FDCR)	1.293 $\pm$ 0.004	47.15 $\pm$ 0.83

Table 9: Input-cache utilization during harness evolution on AgentBench.

Dataset	Method	Cached Tokens	Total Input Tokens	Cache Hit Rate (%)
AgentBench	Self-Evolution	4,782,464	5,175,008	92.41
	ECDYSIS (w/o FDCR)	3,926,016	4,244,516	92.50
	ECDYSIS (w/ FDCR)	5,655,552	5,978,778	94.59

Table 10: Immediate post-evolution evaluation of the evolved harnesses on AgentBench with Qwen3-8B.

Dataset	Method	AVG (%)	Pass@3	Pass^3	Final Evaluation Tokens
AgentBench	Self-Evolution	90.00	90.00	90.00	4,106,142
	ECDYSIS (w/o FDCR)	91.67	100.00	85.00	3,851,561
	ECDYSIS (w/ FDCR)	93.33	95.00	90.00	4,532,939

Table 11: Harness self-evolution API costs on AgentBench.

Dataset	Method	Training Evaluation	Evolution	Total
AgentBench	Self-Evolution	\$0.290	\$1.740	\$2.030
	ECDYSIS (w/o FDCR)	\$0.272	\$1.239	\$1.511
	ECDYSIS (w/ FDCR)	\$0.320	\$1.513	\$1.833

Table 12: Training time (s) for the three harness self-evolution methods on AgentBench.

Dataset	Self-Evolution	ECDYSIS (w/o FDCR)	ECDYSIS (w/ FDCR)	Speedup
AgentBench	1,955.3	1,136.6	1,890.5	1.72 × / 1.03 ×

Table 13: Data-efficient harness evolution on the τ^2 -Retail evaluated with Qwen3-32B. Harnesses are evolved using Qwen3-8B. The Full setting uses the complete training set, whereas Reduced and Random each use five training failures, corresponding to one-quarter of the full training set.

Method	AVG (%)	Pass@3	Pass^3	Training Cost
Self-Evolution	65.00 ± 5.00	75.00	55.00	\$8.484
ECDYSIS (Full)	75.00 ± 0.00	85.00	60.00	\$5.763
ECDYSIS (Reduced)	71.67 ± 7.64	90.00	45.00	\$0.291
ECDYSIS (Random)	65.00 ± 8.66	80.00	45.00	\$0.505